Advanced Data Management
Quiz Refresher: Chapter 5, Questions

Query optimization in the mock's format: 8 quick checks, 3 problem situations that end in a decision, one real plan to annotate, 1 of 2 discussion tickets; answers in the separate answer sheet

Advanced Data Management

2026-10-29

How to use this document

Same shape as Mock Exam v2, same database, only the chapter changes: every question here is chapter 5 (statistics, selectivity, cost model, join ordering, reading plans). Use the mock's formula sheet; nothing else is needed. Grading follows the mock: a wrong final number with the right formula earns most of the points, a bare number earns little, so write the formula, then the numbers, then the one sentence that quotes your number. Budget 20 minutes for Part A, 35 for B, 30 for C, 15 for D. The answer sheet is a separate file; try the whole paper before opening it.

Every situation uses `adm_lab`: `customers` 10,000 rows / 73 pages, `products` 200 rows / 2 pages, `orders` 500,000 rows / 3,185 pages (about 157 rows per page), PostgreSQL 16, 8 KB pages. Statistics are fresh unless a question says otherwise. `amount` is uniform between 5 and 105 and its histogram has 10 equi-depth buckets with bounds $5, 15, 25, \ldots, 105$. `cust_id` has 10,000 distinct values and `product_id` 200, neither with a most-common-values list. The planner's estimate for `order_date >= '2025-12-01'` alone is 43,388 rows (8.68%), as in the mock.

Part	Content	Chapter 5 sections	Points
A	8 quick checks, letter plus one line	2, 3, 4, 5, 6, 7	24
B	3 problem situations	4–5 (access path), 4 and 7 (correlation), 5–6 (join order)	30
C	one real plan, 6 questions	3, 4, 7	26
D	1 of 2 tickets	5 and 7	20

Part A: Quick checks, circle one and justify in one line (24 pts)

Three points each: one for the letter, two for the justification (a number, a formula or one sentence).

A1. How many rows does the planner estimate for `WHERE cust_id = 42` on `orders`?
(a) 5 (b) 50 (c) 500 (d) 10,000

A2. Estimated fraction of `orders` kept by `WHERE amount BETWEEN 20 AND 40`?
(a) 10 % (b) 20 % (c) 25 % (d) 40 %

A3. `EXPLAIN SELECT * FROM orders;` with no `WHERE`. Which total cost does the Seq Scan print?
(a) 3,185 (b) 8,185 (c) 9,435 (d) 12,740

A4. A node shows `rows=20` and `actual rows=20000`. The q-error is:
(a) 0.001 (b) 1,000 (c) 19,980 (d) 1, the plan ran fine

A5. `WHERE country = 'MA' AND city = 'Casablanca'` on `customers`, and every Casablanca customer is in MA. Compared with the truth, the planner's estimate is:
(a) too high (b) too low (c) exact, `ANALYZE` saw the data (d) it depends on the index

A6. A query joins 5 tables. Number of left-deep join orders, and number of non-empty subsets System-R dynamic programming memoises:
(a) 5 and 5 (b) 120 and 31 (c) 120 and 120 (d) 32 and 120

A7. Which of these is a *logical* rewrite rather than a physical choice?
(a) hash join instead of merge join (b) pushing `WHERE o.amount > 100` below the join (c) index scan instead of seq scan (d) building the hash on `customers`

A8. `orders` grew to 1,500,000 rows over a year; autovacuum is off. The planner still prints `rows=500000` for a full scan. Which failure, and what is the first command?
(a) correlated columns, `CREATE STATISTICS` (b) stale statistics, `ANALYZE orders` (c) skew, raise the statistics target (d) wrong costs, lower `random_page_cost`

Part B: Problem situations, calculate then decide (30 pts)

Use the formula sheet. Each situation ends with a decision: answer it in one or two sentences that use your numbers.

Problem: B1 "Just add an index on amount" [10 pts]

The finance report runs `SELECT sum(amount) FROM orders WHERE amount > 95`. A colleague created `CREATE INDEX ON orders (amount)` and is surprised the plan still shows a Seq Scan. In `pg_stats` the correlation of `amount` is 0.002. The B$^+$-tree has 3 levels.

(a) Selectivity of the predicate from the histogram, estimated rows, and the number of pages those rows would occupy if they were packed together. [3 pts]

(b) Cost of the Seq Scan with its filter (three terms). Cost of a plain Index Scan, counting the index descent and one random heap page per matching row, plus the per-row CPU term. [4 pts]

(c) The rule $4sN < B$: below which selectivity would the plain Index Scan win here, and how many rows is that? Which value of `amount` roughly corresponds to it? Decide: is the colleague's index useful for this report, and why does the correlation matter? [3 pts]

Start with: "The predicate keeps ... of the histogram, so ..."

Problem: B2 The Casablanca report is slow [10 pts]

`customers.country` has 5 distinct values and an MCV list where `'MA'` has frequency 0.40. `customers.city` has 200 distinct values and no MCV list. In reality 3,000 customers live in Casablanca and all of them are in MA. The report is

```
SELECT c.id, sum(o.amount) FROM customers c JOIN orders o ON o.
    cust_id = c.id
WHERE c.country = 'MA' AND c.city = 'Casablanca' GROUP BY c.id;
```

`orders` has a 3-level B$^+$-tree on `cust_id`; every customer has about 50 orders, each on a different heap page. Filtered `customers` fits in one page.

(a) Planner's estimate for the filtered `customers`, the true count, and the q-error. Which of the three failures is this? [3 pts]

(b) Estimated and true size of the join output. What happens to the q-error on the way up? [2 pts]

(c) With the formula sheet, cost in page I/O of an index nested loop (customers outer) and of a hash join, first with the *estimated* 20 customers, then with the *true* 3,000. Which plan does the planner pick, and how far off is its real cost from the alternative? [3 pts]

(d) Will `ANALYZE customers` fix it? Give the fix that will, in two SQL statements, and say what the estimate becomes. [2 pts]

Start with: "The planner multiplies ... by ..., so it believes ..."

Problem: B3 Three tables, which join first? [10 pts]

```
SELECT c.id, sum(o.amount)
FROM customers c JOIN orders o ON o.cust_id = c.id
                 JOIN products p ON p.id = o.product_id
WHERE c.country = 'MA' AND p.category = 'books'
GROUP BY c.id;
```

`country = 'MA'` has selectivity 0.40; `category = 'books'` keeps 10 of the 200 products.

(a) Estimated rows of each input after selection pushdown: filtered `customers`, filtered `products`, `orders`. [2 pts]

(b) For each of the three possible first joins, the estimated intermediate size: σ(products) $\bowtie$ orders, σ(customers) $\bowtie$ orders, σ(products) $\times$ σ(customers). Then the size of the final result, computed from two different orders to check it. [4 pts]

(c) Which first join does the planner prefer and why? Does System-R dynamic programming even consider the third one? [2 pts]

(d) The SQL writes `customers` first. Why may the planner start with `products` anyway, and which setting would have frozen the written order if it were 1? [2 pts]

Start with: "After pushdown the inputs are ..., so joining ... first produces only ..."

Part C: Read a real plan (26 pts)

Problem: Ticket REP-31 "The customer page is fine, but the estimate is way off" [26 pts]

The customer page shows the top five products a customer bought in December. A developer noticed the planner's row estimate and asks whether the plan is wrong. The index `idx_orders_cust` is a 3-level B$^+$-tree on `orders(cust_id)`; the correlation of `cust_id` is 0.001.

```
SELECT p.name, sum(o.amount) AS spent
FROM orders o JOIN products p ON p.id = o.product_id
WHERE o.cust_id = 42 AND o.order_date >= '2025-12-01'
GROUP BY p.name ORDER BY spent DESC LIMIT 5;
```

```
Limit  (cost=219.07..219.08 rows=4) (actual rows=5 loops=1)
  ->  Sort  (cost=219.07..219.08 rows=4) (actual rows=5 loops=1)
        Sort Key: (sum(o.amount)) DESC
        Sort Method: top-N heapsort  Memory: 25kB
        ->  HashAggregate  (cost=218.97..219.01 rows=4) (actual rows=41 loops=1)
              Group Key: p.name
              ->  Hash Join  (cost=6.50..218.95 rows=4) (actual rows=45 loops=1)
                    Hash Cond: (o.product_id = p.id)
                    ->  Index Scan using idx_orders_cust on orders o
                          (cost=0.42..212.88 rows=4) (actual time=0.031..0.512
    rows=45 loops=1)
                          Index Cond: (cust_id = 42)
                          Filter: (order_date >= '2025-12-01'::date)
                          Rows Removed by Filter: 7
                          Buffers: shared hit=55
                    ->  Hash  (actual rows=200 loops=1)
                          Buckets: 1024  Batches: 1  Memory Usage: 17kB
                          ->  Seq Scan on products p  (cost=0.00..4.00 rows=200)
    (actual rows=200 loops=1)
                                Buffers: shared hit=2
```

(Execution time 0.702 ms.)

1. Draw the plan as a tree (7 nodes). For each node say whether it must read all its input before producing its first row. [5 pts]
2. Where does `rows=4` on the Index Scan come from? Show the two factors and the assumption that joins them. [5 pts]
3. q-error of the Index Scan. Which of the three estimation failures is at work, and would `ANALYZE` or `CREATE STATISTICS (dependencies) ON cust_id, order_date` fix it? [4 pts]
4. Explain `Buffers: shared hit=55` with numbers. Why one heap page per order? [4 pts]
5. A colleague proposes `CREATE INDEX ON orders (cust_id, order_date)`. What changes in the plan lines and in `Buffers`? Does the estimate change? Would an index on `order_date` alone be a better idea? [5 pts]
6. The estimate is off by an order of magnitude yet the query runs in 0.7 ms. Why is the error harmless here, and describe a plan shape where the same factor would be a disaster. [3 pts]

Start with: "The Index Cond keeps about ... rows because ..., then the Filter keeps ..."

Part D: Discussion, choose 1 of 2 (20 pts)

Half a page. A good answer: claim → mechanism → number → consequence.

Problem: Ticket D1 "Our disks are SSDs, set random_page_cost = 1 everywhere" [20 pts]

A DBA proposes `random_page_cost = 1.0` on every server because "random reads cost the same as sequential ones on SSD". Explain what the parameter does in the cost model, how the index-versus-scan threshold moves on `orders` (use rows per page), which kinds of plans will flip and in which direction, what could go wrong, and what you would measure before and after. Start with: "The cost model charges ..."

Problem: Ticket D2 "The planner picked a bad plan, we need optimizer hints" [20 pts]

After a report ran 40 minutes with a nested loop over millions of rows, a developer wants to force plans with hints, as other databases allow. Explain why the plan was probably chosen (which estimate, which failure), the order in which PostgreSQL expects you to fix it (statistics, then cost parameters, then memory, then `enable_*` for experiments, then an extension), and why a hint that is right today can be wrong next year. Start with: "A 40-minute nested loop is almost always a cardinality error: ..."